

# Choix des technologies

## Analyse par domaine

### I. Technologies front

| Critère                         | Flutter | Expo (React Native) |
|---------------------------------|---------|---------------------|
| Partage de code (Web/Mobile)    | ★★★★★   | ★★★★★★              |
| Performance Native              | ★★★★★★  | ★★★★★               |
| Écosystème IA (SDK/Stream)      | ★★★     | ★★★★★★              |
| Flexibilité Design (UI/UX)      | ★★★★★   | ★★★★★★              |
| Facilité d'apprentissage        | ★★★     | ★★★★★               |
| Cartographie Interactive        | ★★★★★   | ★★★★★★              |
| Projet long terme / Scalabilité | ★★★★★   | ★★★★★★              |
| Proposition de choix            | ✗       | ✓                   |

#### a) Structure et partage de code

Expo utilise le même moteur de rendu pour le web et pour le mobile avec un seul routeur. Expo permet de faire du natif, tout en conservant un DOM propre (avantage pour le SEO).

### **b) Ecosystème IA**

Les outils comme Vercel AI SDK ou les intégrations Langchain sont développés en "JS First". Expo permet d'accéder aux bibliothèques utilisées par les plus grosses entreprises IA (on peut en déduire que ce sont donc des outils fiables).

### **c) Design et ergonomie**

Flutter est adapté pour dessiner des composants complexes, mais React Native (avec NativeWind/Tailwind) est devenu plus agile pour créer des fils d'actualité fluides et des interfaces de profil modernes sans la rigidité du langage Dart.

### **d) Apprentissage**

ReactNative est parfait pour les personnes déjà familières à React.js, plus simple à prendre en main. Choisir Flutter voudrait dire apprendre un nouveau langage (Dart).

### **e) Cartographie**

Pour les itinéraires, Mapbox dispose d'un vaste support sur ReactNative offrant des performances de zoom et de manipulation de calques supérieures à d'autres librairies.

### **f) Outils et librairies**

- UI/UX : NativeWind (équivalent de Tailwind pour mobile)
- Gestion de requêtes : Tanstackquery (exemple : scroll infini)
- Cartographie : Mapbox (optimisation des itinéraires, actualisation des plans en temps réel...), react-native maps
- Intégration du chatbot IA : AI SDK UI Vercel
- Maps : react-native maps
- AI SDK UI (Vercel) : helps to create chatbox, load messages etc

## II. Technologies back

| Critère              | NestJS | Express | Fastify | Adonis |
|----------------------|--------|---------|---------|--------|
| Structure            | ★★★★★★ | ★       | ★★      | ★★★    |
| Scalabilité          | ★★★★★★ | ★★      | ★★★     | ★★★    |
| Facilité débutant    | ★★★    | ★★★★★   | ★★      | ★★     |
| TypeScript           | ★★★★★★ | ★★      | ★★★★★   | ★★★    |
| Écosystème           | ★★★★★★ | ★★★★★★  | ★★★     | ★★     |
| Projet long terme    | ★★★★★★ | ★★      | ★★★     | ★★★    |
| Proposition de choix | ✓      | ✗       | ✗       | ✗      |

### a) Structure

NestJS t'impose une architecture propre dès le début : modules, contrôleurs, services, DTO, pipes, guards, interceptors...

- Résultat : code lisible, séparation claire des responsabilités, et surtout facile à maintenir quand le projet grossit.
- Comparaison : Express/Fastify sont très flexibles, mais cette liberté mène vite à une architecture "au feeling" si tu ne cadres pas toi-même.

### b) Scalabilité

NestJS est conçu pour passer à l'échelle :

- Architecture modulaire (tu peux découper par domaines : Auth, Users, Orders, etc.)
- DI (injection de dépendances) native → facilite les évolutions et refactors

- Bonnes pratiques prêtes pour des projets d'équipe (patterns, conventions)
- Comparaison : Express/Fastify peuvent scaler, mais c'est toi qui dois mettre en place la discipline et les patterns.

#### **c) Facilité débutant**

Oui, Nest a une courbe d'apprentissage (DI, modules, decorators).

Mais c'est un investissement rentable :

- Manière de faire "pro"
- Evite de construire une architecture bancale puis refactor plus tard
- Sur long projet, le gain dépasse largement le coût initial

#### **d) TypeScript**

NestJS est TypeScript-first (pas "compatible TS", mais pensé pour).

- Typage fort partout (DTO, services, validation, contrats)
- Moins de bugs à l'exécution, plus d'auto-complétion, refactor plus safe
- Comparaison : Express + TS marche bien, mais on sent souvent le côté "adapté" plutôt que "natif".

#### **e) Écosystème**

NestJS a un énorme écosystème orienté entreprise :

- Auth (JWT, OAuth), validation, Swagger/OpenAPI, WebSockets, microservices, queues, etc.
- Intégrations fréquentes : ORM, cache, config, tests...
- Comparaison : Express a beaucoup de packages, mais c'est plus "à assembler". Nest te donne un cadre + des intégrations propres.

#### **f) Projet long terme**

Pour un projet qui dure, NestJS apporte :

- Conventions stables → nouveaux devs onboard rapidement
- Maintenabilité + testabilité (DI + architecture)
- Cohérence globale du codebase
- Comparaison : sur du long terme, les frameworks "non-opinionated" (Express/Fastify) peuvent devenir hétérogènes si les règles ne sont pas strictes.

## **III. Technologies IA**

| Brique                | Décision retenue                     | Justification                                                                                                                        |
|-----------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| LLM                   | Mistral 7B via Ollama (local)        | Coût nul, exécution locale, contrôle total des données, conformité RGPD, cohérent avec les contraintes MVP (budget, équipe réduite). |
| Architecture chatbot  | RAG                                  | Données sous contrôle, contexte dynamique, pas de fine-tuning                                                                        |
| Recommandation        | Hybride content-based + collaboratif | Gestion du cold start via profil psychologique (Profil vectoriel personnalisé), enrichissement progressif                            |
| Données contextuelles | Open-Meteo + Google Trends           | Gratuites ou très peu coûteuses, fiables                                                                                             |

#### a) Collecte et ingestion des données sociales

##### 1. Sources de données pour l'analyse des tendances

| Plateforme         | API disponible ?         | Accès gratuit | Usage dans Triply                         |
|--------------------|--------------------------|---------------|-------------------------------------------|
| Google trends      | Pytrends / Google trends | Gratuit       | Identification des destinations en hausse |
| Open Data Tourisme | DataTourisme API         | Gratuit       | Lieux, activités, POI officiels           |
| Données Internes   | Base triply              | Gratuit       | Popularité basée sur clics et favoris     |

### ⚠ Risque juridique — Scraping

Le scraping sans autorisation explicite viole les CGU de la plupart des plateformes et peut constituer une infraction. Le scraping peut être envisagé ultérieurement via des partenariats ou des outils tiers certifiés (Apify, Bright Data).

## 2. Pipeline d'ingestion recommandé

Étape 1 — Collecte : Requêtes planifiées vers Google Trends et synchronisation régulière des données Open Data Tourisme

Étape 2 — Normalisation : Transformation des données en schéma unifié (destination, catégorie, score de popularité normalisé)

Étape 3 — Stockage : Enregistrement en base PostgreSQL avec indexation par région, catégorie et période

Étape 4 — Signaux : Calcul d'un TrendScore pondéré combinant tendances Google et popularité interne

### 3. Synthèse

| Source      | API                           | Données fournies                         | Gratuité                                                                    |
|-------------|-------------------------------|------------------------------------------|-----------------------------------------------------------------------------|
| Météo       | Open-Meteo                    | Prévisions 16j, données historiques      | 100% gratuit                                                                |
| Événements  | Ticketmaster API / Eventbrite | Concerts, festivals, événements locaux   | Gratuit (quota limité - clé api requise) / (5000req/jour)                   |
| Actualités  | NewsAPI.org                   | Articles de presse par mot-clé et pays   | Gratuit quota limité extension payante possible (100 req/jour)              |
| Lieux / POI | Foursquare Places API         | Points d'intérêt, restaurants, activités | 10,000 req gratuite (free tiers limité, conditions susceptible d'évolution) |

#### b) Moteur de recommandation

Le moteur de recommandation constitue la brique centrale de Triply. Pour le MVP, l'objectif n'est pas de construire un système complexe, mais un moteur : pertinent dès l'inscription, simple à maintenir et évolutif, etc.

#### 1. Stratégie de gestion du cold start

Problème du cold start : Le principal défi initial est le cold start : recommander sans historique utilisateur.

#### Phase 1 — Démarrage (0 à 5 interactions)

S'appuyer exclusivement sur le profil psychologique (Profil vectoriel personnalisé)

Calcul de similarité cosinus entre profil utilisateur et destination

Utilisation des préférences déclarées (budget, durée, type de voyage, etc.)

Recommandations basées uniquement sur la similarité de contenu (content-based filtering)

#### Phase 2 — Enrichissement (5 à 20 interactions)

Intégration progressive des feedbacks implicites (clics, temps passé, ajouts en favoris)

Pondération croissante du comportement réel

Activation partielle du filtrage collaboratif avec les profils similaires

#### Phase 3 — Personnalisation complète (20+ interactions)

Introduction du filtrage collaboratif

Similarité entre utilisateurs

Diversification intelligente des recommandations

## **2. Architecture hybride du moteur**

| Composant          | Technique                                             | Poids MVP     | Librairie / Outil       |
|--------------------|-------------------------------------------------------|---------------|-------------------------|
| Profil utilisateur | Vecteur simplifié (préférence + profil psychologique) | Fort          | numpy                   |
| Content-based      | Similarité cosin                                      | Fort          | scikit-learn, numpy     |
| Collaboratif       | Similarité                                            | MVP simplifié | Similarité cosin        |
| Contextuel         | Boost pondéré selon météo, events, tendances          | Moyen         | Règles pondérées custom |
| Diversification    | MMR (Maximal Marginal Relevance)                      | Faible        | Custom                  |

### 3. Représentation vectorielle du profil utilisateur

Le profil utilisateur est représenté sous forme d'un vecteur de caractéristiques combinant :

Préférences de voyage : budget, durée, type de voyage

Axes de personnalités : profil psychologique (Profil vectoriel personnalisé)

Feedback (clics, favoris, interactions)

Ces valeurs sont indicatives et seront ajustées empiriquement lors des premières itérations.

### 4. Métriques d'évaluation du moteur

Sans métriques définies dès le départ, impossible de savoir si le moteur s'améliore. Voici les KPIs à instrumenter dès le MVP :

| Métrique                    | Définition                                               | Objectif MVP                            | Comment mesurer                          |
|-----------------------------|----------------------------------------------------------|-----------------------------------------|------------------------------------------|
| CTR<br>(Click-Through Rate) | % de recommandations cliquées                            | À observer et améliorer progressivement | Tracking des interactions UI             |
| Precision @k                | Pertinence sur les K premières recommandations affichées | À monitorer via feedback utilisateur    | Like, dislike, ajouter en favoris        |
| Diversité                   | Variété des destinations suggérées                       | Eviter la répétition excessive          | Analyse de distribution des catégories   |
| Satisfaction                | Note globale après interaction du voyage                 | Suivi qualitatif                        | Enquête post-voyage in-app               |
| Cold start drop-off         | % d'abandon lors de la création du compte                | Identifier les points de friction       | Analytics d'entonnoir (funnel analytics) |

### c) Assistant conversationnel – Chatbot IA

Le chatbot est l'interface principale entre l'utilisateur et le moteur de recommandation. Son rôle va au-delà de la simple conversation : il collecte des préférences, explique les suggestions, gère les objections et accompagne tout le parcours utilisateur.

#### 1. Stratégie LLM retenue pour le MVP

Dans le cadre d'un MVP à périmètre restreint, la priorité est de minimiser les coûts tout en gardant le contrôle total sur les données. La stratégie retenue repose sur des modèles open-source légers, exploitables localement en développement et via des APIs gratuites en production initiale.

#### 2. Comparatif des modèles LLM

| Modèle       | Via            | Contexte    | Coût MVP | Qualité | Recommandation     |
|--------------|----------------|-------------|----------|---------|--------------------|
| Mistral 7B   | Ollama (local) | 32k tokens  | 0€       | Bon     | ✓ Choix MVP        |
| LLaMA 3.2 3B | Ollama (local) | 128k tokens | 0€       | Correct | Option très légère |
| LLaMA 3.1 8B | Ollama (local) | 128k tokens | 0€       | Bon     | Bon équilibre      |

### 3. Architecture RAG — Détail technique

L'architecture RAG (Retrieval Augmented Generation) permet d'injecter des données contextuelles dans chaque requête LLM sans fine-tuning. C'est le choix optimal pour un MVP : rapide à implémenter, maîtrisé juridiquement, et évolutif.

Flux RAG — Étapes détaillées :

1. Requête utilisateur reçue par le chatbot
2. Extraction des entités clés (destination, dates, budget, activités souhaitées)
3. Requête vectorielle dans pgvector → top 5 destinations pertinentes
4. Récupération du profil utilisateur (pseudonymisé) + contexte (météo, événements)
5. Construction du prompt augmenté : [System prompt] + [Données contextuelles] + [Historique conversation] + [Requête utilisateur]
6. Appel du modèle LLM Local (Ollama) → génération de réponse
7. Logging pseudonymisé pour amélioration continue

### 4. Gestion du contexte conversationnel

Fenêtre de contexte : conserver les 10 derniers échanges pour maintenir la cohérence

Résumé de session : après 10 échanges, compresser l'historique via le LLM Local (Ollama) pour économiser les tokens

Mémoire persistante : stocker en base les préférences extraites de la conversation (ex : 'préfère la mer à la montagne')

Gestion des intentions : classifier les messages (demande de reco, question pratique, feedback, hors-sujet)

**d) Conformité RGPD et sécurité des données**

**1. Données sensibles identifiées**

| Donnée                                                  | Sensibilité                      | Traitement recommandé                                       |
|---------------------------------------------------------|----------------------------------|-------------------------------------------------------------|
| Profil psychologique<br>(Profil vectoriel personnalisé) | Donnée de personnalité           | Stockée en base, jamais envoyée brute au LLM Local (Ollama) |
| Historique de navigation                                | Comportemental                   | Pseudonymisation (UUID interne), Durée de vie 90 jours      |
| Données sociales collectées                             | Publiques agrégées               | Agrégation par thème, aucun ID utilisateur stocké           |
| Conversations chatbot                                   | Contenu potentiellement sensible | Pseudonymisation, pas de logging des données personnelles,  |

|              |                      |                                                                             |
|--------------|----------------------|-----------------------------------------------------------------------------|
| Localisation | Donnée sensible RGPD | Collecte avec<br>consentement explicite,<br>granularité ville<br>uniquement |
|--------------|----------------------|-----------------------------------------------------------------------------|

## 2. Mesures techniques obligatoires

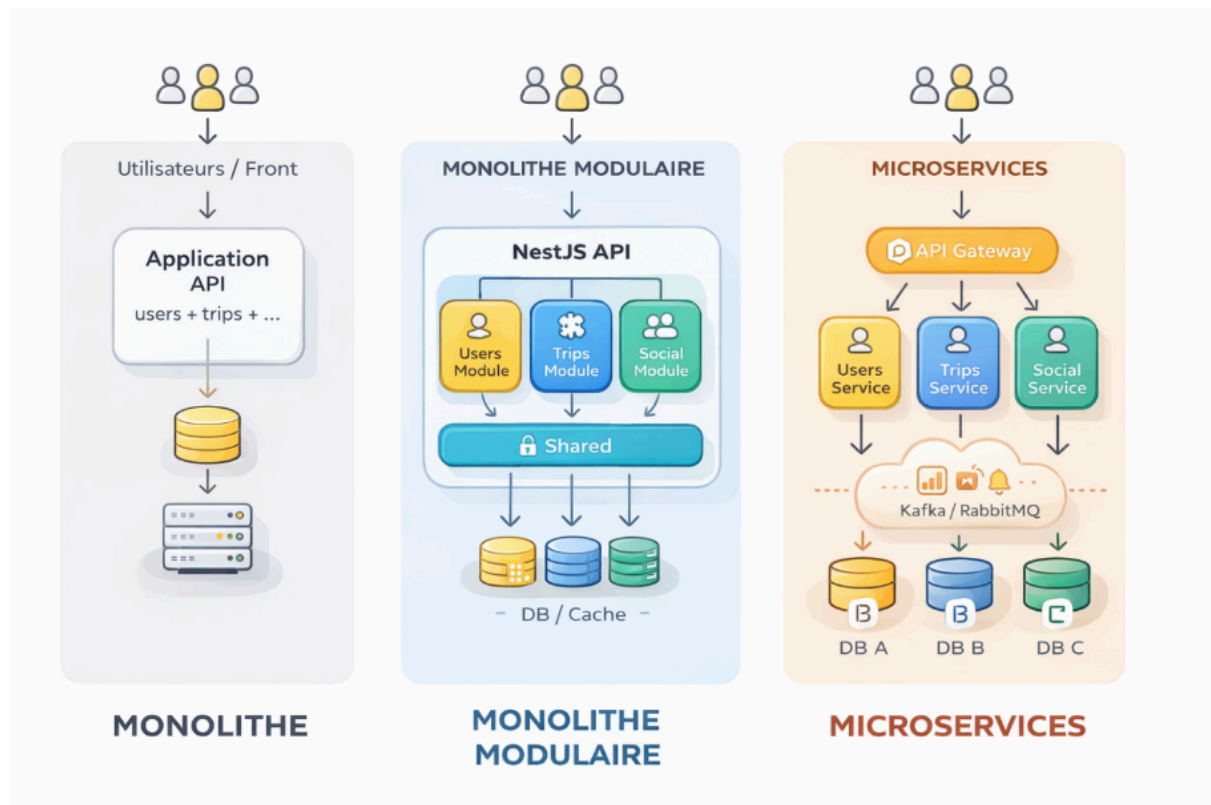
Chiffrement au repos et en transit pour toutes les données utilisateurs

Pseudonymisation systématique avant injection dans le LLM : remplacer noms/emails par des UUID

Droit à l'effacement : pipeline de suppression complète des données utilisateur sur demande

Registre de traitement documenté dès le lancement

## IV. Architecture



| Critère                    | Monolithe | Monolithe modulaire | Microservices |
|----------------------------|-----------|---------------------|---------------|
| Structure                  | ★★★★      | ★★★★★★              | ★★★★★★        |
| Scalabilité                | ★★        | ★★★★★★              | ★★★★★★        |
| Facilité débutant          | ★★★★★★    | ★★★★★★              | ★★            |
| Standardisation & contrats | ★★★★      | ★★★★★★              | ★★★★★★        |
| Écosystème / outillage     | ★★★★★★    | ★★★★★★              | ★★★★★★        |

|                                   |     |       |       |
|-----------------------------------|-----|-------|-------|
| Projet long terme                 | ☆☆☆ | ☆☆☆☆☆ | ☆☆☆☆☆ |
| Proposition de choix (entreprise) | ✗   | ✗     | ✓     |

Le Microservices semble être le meilleur choix :

- Meilleur ratio valeur/complexité : architecture claire, tests, conventions, déploiement simple.
- Compatible avec une évolution progressive :
  - microservice possible si un domaine le justifie

**a) Proposition d'architecture :**

- api : NestJS (core)
- postgres : données métier
- redis : cache + BullMQ
- mongo : chatbot + logs
- ai-service : (optionnel) microservice IA (Python/FastAPI)
- qdrant : base vectorielle pour RAG
- Optionnel : adminer / pgadmin (UI Postgres)

**b) DevOps**

- Conteneur pour chaque service :
  - Front
  - Back
  - Chaque BDD
- Watch : rebuild à chaud
- Docker compose : lancer en une seule commande
- Phase de la CI :
  - Lint
  - Sonar
  - Test
- Optionnel : Makefile
- Optionnel : K8s
- Optionnel : Terraform ? Quel provider ?